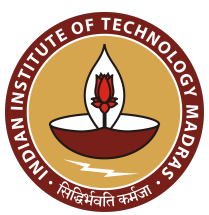
Version Control and Collaborative Project Management

Saurav Samantaray

Department of Mathematics

Indian Institute of Technology Madras

October 18, 2026



Why do we need version control?

Let's say a developer has the following files:

- `solver.py`
- `solver_final.py`
- `solver_final_updated.py`
- `solver_final_revised2.py`
- etc. You get the idea !!!

Few questions we should ask

- Which version contains the correct algorithm?
- Can we identify exactly what changed between Tuesday and Thursday?
- How do we recover a working version after introducing a bug?
- What happens when two someone else edits the same function?

Why do we need version control?

- Saving multiple copies may seem convenient, but it quickly becomes confusing.
- We need a systematic way to track changes, understand their history, and recover earlier work.

We need version control, not a collection of filenames.

What Is Version Control?

- Keeps track of changes to text files line-by-line over time;
- tracks change between any two versions;
- enables to recover or revert to an earlier version.
- back up and distribute copies of files;
- collaborate on projects.

Version control records the evolution of a project.

Instead of relying on separate files, we record meaningful snapshots of our work.



Creating Our First Git Repository

We will develop a small Python library for numerical methods.

```
mkdir numerical-toolbox
cd numerical-toolbox
git init
```

What do these commands do?

- `mkdir`: creates the project directory.
- `cd`: moves into the project directory.
- `git init`: initializes a new Git repository in the directory.

Before `git init`

An ordinary directory containing files.

After `git init`

The directory is now a Git repository, ready to track project history.

Creating Our First Git Repository

What has happened yet?

- Git has created the repository's internal metadata, traditionally stored in a hidden `.git` directory.
- Our Python files have not yet been recorded in the history well we haven't even created them yet !!!.

Initializing a repository does not automatically track (commit) every file.

Run `git status`

First file tracking

- create `solver.py`
- Run `git status`
- `Untracked files:`
  `(use "git add <file>..." to include in what will be committed)`
  `solver.py`

Creating Our First Git Repository

Untracked : A newly created, untracked file
What does “untracked” mean?

- The file exists in the working directory.
- Git has not yet been instructed to include it in version control.
- It does not appear in the repository’s commit history.

If we delete solver.py now, can Git recover it from a previous commit?
No. The file has not been committed, so Git has no committed version of it to recover.

Committing (adding to version control) our first file

```bash
git add solver.py
git status
git commit -m "Add initial solver structure"
git log --oneline
```

- "`git add`" new or modified files to the index.
- "`git status`" tells us the state of our repository:
  - which files are untracked and not indexed;
  - which files are modified with respect to the index / staging area;
  - which files are staged (indexed) and are ready to be committed;
- "`git add`" selects changes for the next commit; it does not itself create a commit.
- The staging area lets us choose which changes belong together.

Committing (adding to version control) our first file

- "`git commit`" records the staged snapshot in the repository's history.
- "`git log --oneline`" displays the recorded commits in a compact format.

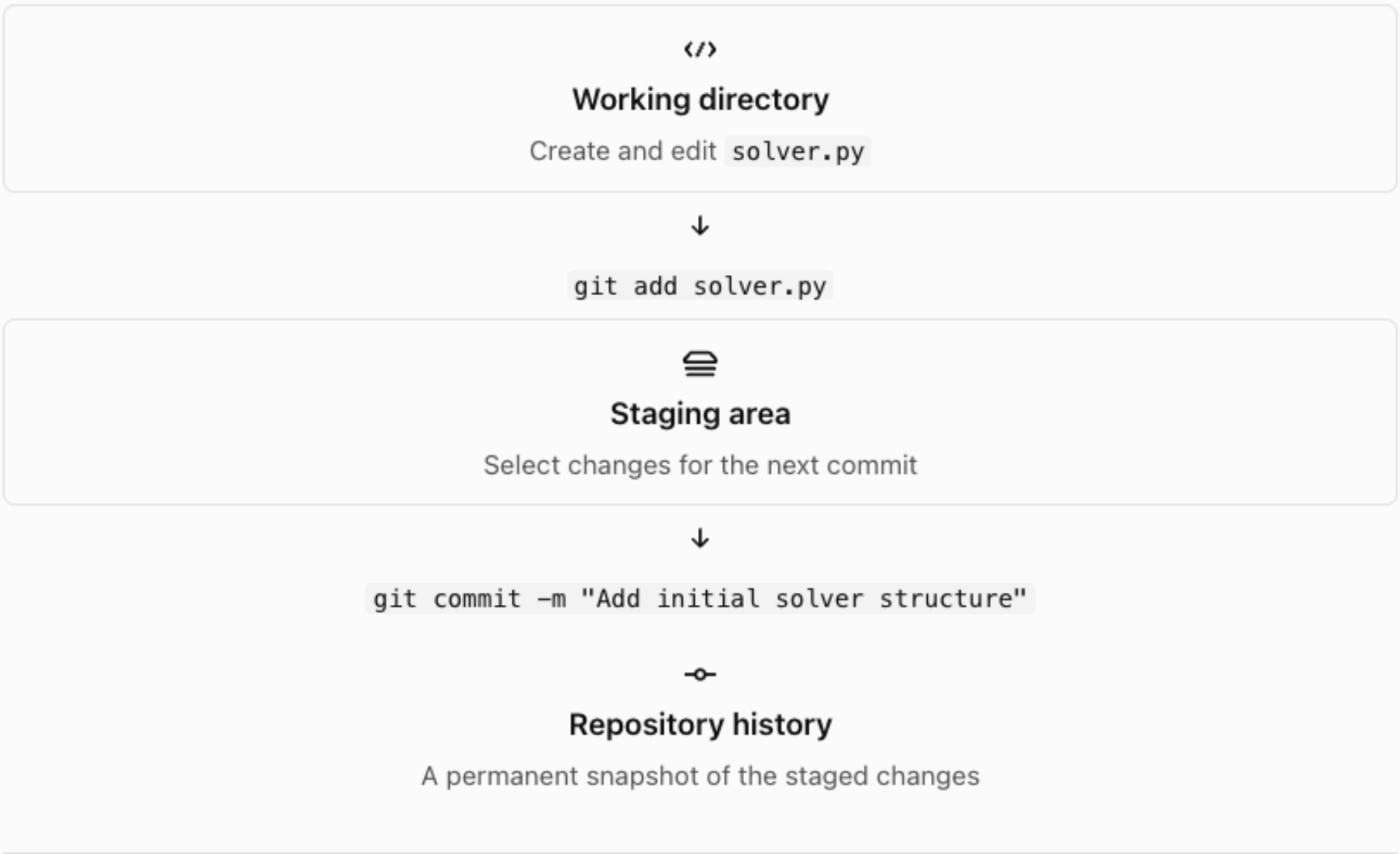
When do we `commit`?

- `commit` is not like `ctrl + s`
- A `commit` is a meaningful checkpoint in the development of the project.

`-m` stands for message. It lets us provide the commit message directly in the command.

- `git commit`: records the staged changes.
- `-m`: specifies the commit message.
- "`Add initial solver structure`": describes what the commit does.

Committing (adding to version control) our first file



What Has Changed? Introducing `git diff`

- Modify solver.py by replacing the placeholder with a simple implementation
- Now run:
  - `git status`
  - `git diff`
- it shows
  - + Lines added or modified
  - - Lines removed or replaced

What does `git diff` show?

It compares the current unstaged changes with the last committed version.

Have we recorded this implementation in Git history yet?

- Run `git diff` before staging. Git lets us inspect exactly what we are about to record.
- Then run `git add solver.py` followed by `git diff --staged` to show the staged changes.

Understanding the Project History

After staging and committing the implementation, run:

```
git add solver.py
git commit -m "Implemented bisection method"
git log --oneline
```

The history now contains two commits:

- Commit 2 - Implemented bisection method
  Recorded the addition of the algorithm
- Commit 1 - Add initial solver structure
  Recorded initial project structure

To inspect a particular commit:

```
git show HEAD
```

`git log` displays the sequence of commits

A repository is not just the current version of a file. It also contains a history of committed snapshots. That history allows collaborators to understand how the project evolved.

Branches: Dev Without Disturbing the Stable Version

- Suppose bisection is working, and we now want to implement Newton's method.
- We want to develop the new feature without directly changing our stable main branch.

```
git switch -c feature/newton-method
```

This creates a new branch and switches to it.




```
git branch
git switch master
git switch feature/newton-method
```

Branches: Dev Without Disturbing the Stable Version

- A branch is a separate line of development, not a separate copy of the entire project that must be managed manually.
- Both branches initially point to the same latest commit.
- New commits on the feature branch can advance independently of main.
- Switching branches changes which version of the project you are working on.

If we discover a bug in bisection while developing Newton's method, why is working on a separate branch useful?

It isolates the new feature from the stable version and makes parallel development easier to manage.

Commit Work on a Feature Branch

- We are currently working on `feature/newton-method`.
- Add Newton's method to `solver.py`.

```
git status
git diff
git add solver.py
git commit -m "Add Newton method"
git log --oneline --all --graph
```

What do we observe?

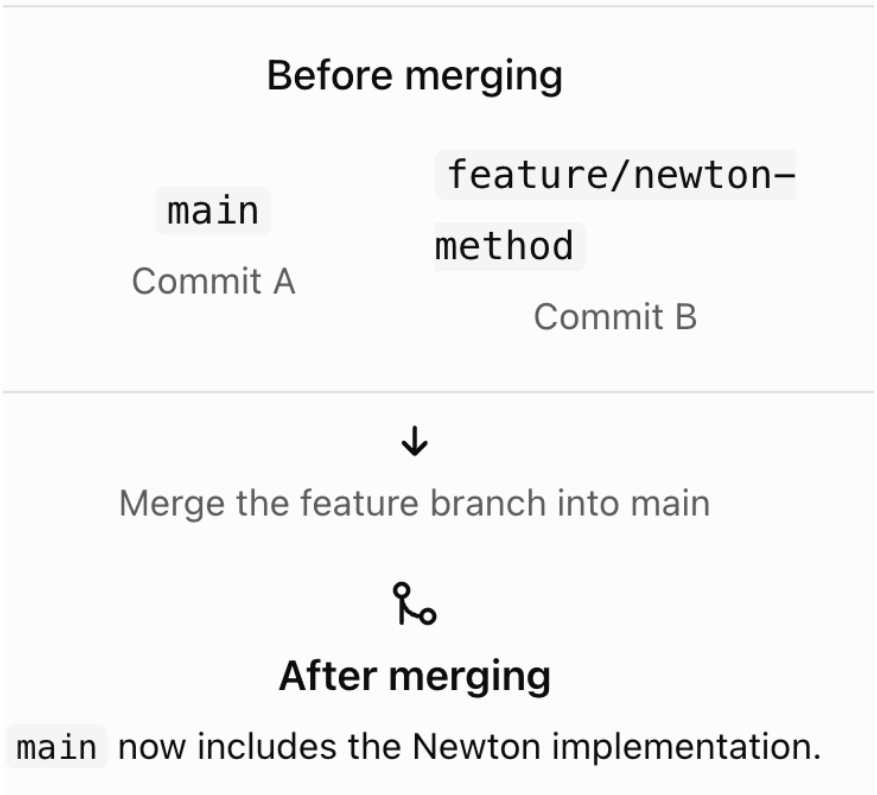
- The new implementation is committed on the feature branch.
- The `master` branch still points to its earlier commit.
- Git records the new work without requiring us to modify `master`.

- A branch lets us develop and commit a feature independently.
- The commit is part of the repository's history, but it is not automatically incorporated into `main`

Merging a Feature into main

- The Newton implementation is ready.
- We now want to integrate it into the stable project.

```
git switch master
git merge feature/newton-method
```



Merging a Feature into main

- A branch isolates development.
- A commit records a change.
- A merge integrates changes from one branch into another.

- Because main has not changed since the feature branch was created, Git can usually integrate this particular change automatically.
- Git may perform a fast-forward merge, moving the main pointer forward without creating a separate merge commit.

Does merging mean that Git has verified Newton's method is mathematically correct?

No. Git integrates changes; correctness still requires tests and review.

What Happens When Changes Conflict?

- Now imagine two developers independently modify the same line in `solver.py`.
- Developer A changes the function to:

```
def f(x):
    return x**2 - 2
```

- Developer B changes it to:

```
def f(x):
    return x**2 - 3
```

- Both changes may be valid for different problems, but they cannot both define the same function in the same way.
- When Git cannot automatically reconcile the changes, a merge conflict occurs.

What Happens When Changes Conflict?

Let's assume your branches are:

- `master:  contains return x**2 - 2`
- `feature/another-change:  contains return x**2 - 3`

Switch to master

```
git switch master
```

Check the current version of the file:

```
cat solver.py
```

- we see the version committed on `master`
- Now Merge the other branch

```
git merge feature/another-change
```

- Because both branches changed the same line differently, Git should report a conflict

What Happens When Changes Conflict?

Inspect the conflict

```
git status
```

When we open `solver.py` in our editor. We should see something like:

```
def f(x):
<<<<<<< HEAD
    return x**2 - 2
=======
    return x**2 - 3
>>>>>>> feature/another-change
```

Here:

- `HEAD` refers to the version in your current branch, main.
- The section below `=======` is the incoming version from `feature/another-change`.
- Git is asking you to decide which version should remain.

What Happens When Changes Conflict?

Resolve the conflict

- For this demonstration, suppose the intended problem is $f(x) = x^2 - 2$.
- Edit the file to remove the conflict markers and retain the correct implementation.
- Save the file.

Tell Git that the conflict is resolved

```
git add solver.py
git status
git commit -m "Resolve merge conflict"
```

We can inspect the resulting history:

```
git log --oneline --graph --all
```

From Local Git to Remote Repositories

- So far: Our repository exists on one computer.
- What if another student needs the code?
- What if we want to work from a different computer?
- How do we share our commits?
- A remote repository allows developers to exchange commits. Each developer can still have a complete local repository and work independently

